

The VOID-collapse model in 3D Truchas flow

Implementation and mathematical interpretation

September 12, 2026

Abstract

Truchas provides an experimental model for collapsing small regions of VOID entrained in liquid. It introduces a prescribed negative divergence into the pressure projection in selected mixed liquid/void cells. The corrected velocity subsequently drives volume tracking; when a cell becomes overfilled, the geometric tracker preferentially removes VOID. This note derives the implemented divergence source, explains cell selection and the time-level coupling, and distinguishes the algorithm from a prescribed decay law or a physical gas-compression model.

Source basis. This describes the refactored, mainline 3D flow code at commit `344a7c3a` (full revision `344a7c3a98d5046533585e6fedc87ffee312388b`). The relevant sources are also identical in the 2D worktree at `ba178536`. It does not describe the older v3.0.1 flow implementation or an implementation of collapse in 2D. Equations below are algebraic consequences of the source unless explicitly identified as illustrative assumptions. No simulation was run for this note.

1 Purpose and physical interpretation

The reference manual explains the motivation: volume tracking can leave small fragments of VOID behaving effectively as incompressible bubbles, which persist in the liquid. Enabling `void_collapse` allows a compressive velocity field to develop around these fragments. The manual labels both the model and its relaxation parameter experimental [8].

VOID represents empty space. This model contains no gas equation of state, gas mass, gas energy, bubble pressure law, or surface-tension force. In particular, it does not solve a physical bubble-radius evolution equation. The new term is an empirical modification of the divergence constraint, coupled to the tracker’s special treatment of VOID.

Pure void cells already provide a zero-pressure condition to the ordinary free-surface projection. Collapse adds a mechanism for mixed cells without an immediately adjacent pure void cell. Enabling collapse does not directly delete volume fractions in the projection routine [1, 5].

2 Volume fractions and cell selection

Let cell c have fixed geometric volume V_c . Define

$$F_c = \sum_{\text{real fluid phases and VOID}} f_{m,c}, \quad L_c = \sum_{\text{real fluid phases}} f_{m,c}, \quad (1)$$

$$\alpha_c = F_c - L_c, \quad S_c = 1 - F_c. \quad (2)$$

Here α_c is the VOID fraction of the *whole cell*; it is not normalized by the fluid-accessible fraction F_c . Non-fluid phases are represented by the lumped solid fraction S_c .

Symbol	Source component	Meaning
F_c	<code>props%vof(c)</code>	Fluid fraction including VOID
L_c	<code>props%vof_novoid(c)</code>	Real-fluid fraction excluding VOID
α_c^n	<code>vof_n - vof_novoid_n</code>	VOID fraction at the last accepted state
$\delta\alpha_c$	<code>props%void_delta_cc(c)</code>	Current fraction minus last accepted fraction
λ	<code>void_collapse_relaxation</code>	Dimensionless relaxation factor

The executable classification in `flow_props%update_cc` is as follows [2]. The order of these tests matters.

1. If $F_c < \varepsilon_f$, classify the cell as `solid_t`.
2. Otherwise, if $L_c = 0$, classify it as `void_t`.
3. Otherwise, if $F_c > L_c$, provisionally classify it as `regular_void_t`.
4. Otherwise classify it as `regular_t`.

The internal `cutvof` default is $\varepsilon_f = 0.01$. These are the actual tests in `update_cc`; the introductory description in `flow_domain_types.F90` does not fully describe them.

A second pass changes a provisional `regular_void_t` cell to `regular_t` if *any face neighbor* is `void_t`. Boundary entries with no neighboring cell are skipped. Thus a cell is eligible for collapse precisely when it retains the mixed-cell label and the collapse option is enabled. Both regular cell labels are active in the pressure solve.

This is a local adjacency test, not a connected-component or bubble-size test. A collection of mixed cells can be eligible even when its VOID forms a connected region. Conversely, mixed cells next to a pure void cell are excluded. Solid neighbors do not themselves exclude a mixed cell, and the presence of a physical boundary is not a separate exclusion criterion.

Cell and face labels have different meanings. For a cell, `regular_void_t` identifies the selected mixed state just described. For a face, the same integer label identifies an active-fluid/`void_t` connection, where the pressure on the void side is zero. A collapse-eligible cell need not have any such face; indeed the neighbor test excludes that configuration [2, 1].

3 The implemented projection source

3.1 History-dependent collapse amount

The property update computes

$$\delta\alpha_c = \alpha_c - \alpha_c^n. \quad (3)$$

This is an Eulerian cellwise change. It can reflect transport and whatever other updates have changed the current material/phase distribution; it is not a material derivative or a quantity extracted from the predicted velocity divergence [2, 3].

For an eligible cell define the nonnegative fraction

$$q_c = \min(|\delta\alpha_c|, \lambda\alpha_c). \quad (4)$$

For other cells, or when collapse is disabled, set $q_c = 0$. The mainline implementation expresses this with two sign branches. After the predicted-velocity flux sum has been accumulated, it performs the following operations; the last division acts on the *entire* right-hand side:

```
void = lambda * (props%vof(c) - props%vof_novoid(c))
if (props%void_delta_cc(c) < 0) then
  rhs(c) = rhs(c) + V(c)*max(props%void_delta_cc(c), -void)/dt
end if
if (props%void_delta_cc(c) > 0) then
  rhs(c) = rhs(c) - V(c)*min(props%void_delta_cc(c), void)/dt
end if
rhs(c) = rhs(c)/dt
```

This excerpt abbreviates component names but preserves the signs and scaling in `setup_solver` [1]. Both nonzero branches *subtract* $V_c q_c / \Delta t$ before the final division. Hence the additional pressure-system right-hand side is

$$\Delta b_c = -\frac{V_c q_c}{\Delta t^2}. \quad (5)$$

The two factors of Δt are intentional in this transcription: one converts a fraction change to a volume rate, and the ordinary projection scaling supplies the other.

3.2 Resulting divergence constraint

Let u_f be velocity in the globally oriented face-normal direction, A_f the face area, and $s_{cf} = \pm 1$ convert that direction to the outward normal of cell c . Define integrated divergence

$$(Du)_c = Q_c = \sum_{f \in \partial c} s_{cf} A_f u_f. \quad (6)$$

Positive Q_c denotes net outflow. In the code, the initial flux sum for the projection right-hand side is $-Q_c^*$.

Write the pressure-to-face-normal-gradient operator as G , and let $R = \text{diag}(1/\rho_f)$. With known pressure boundary contributions accounted for, the pressure-correction matrix has the form

$$A = -DRG, \quad A \delta p = -\frac{Du^*}{\Delta t} - \frac{Vq}{\Delta t^2}, \quad (7)$$

where Vq denotes the vector with entries $V_c q_c$. On an ordinary interior face the matrix coefficient is

$$a_f = \frac{\mathbf{d}_f \cdot \mathbf{N}_f}{\rho_f}, \quad (8)$$

using `flow_gradient_coefficients()` for $\mathbf{d}_f$ and the area-weighted normal $\mathbf{N}_f$. The collapse option changes the right-hand side, not these coefficients, for a fixed property state.

The ordinary velocity correction remains

$$u^{\text{new}} = u^* - \Delta t R G \delta p. \quad (9)$$

Combining this with equation (7) gives

$$\frac{Q_c^{\text{new}}}{V_c} = -\frac{q_c}{\Delta t}. \quad (10)$$

Thus eligible cells acquire a prescribed net inflow. Elsewhere the usual zero-divergence target remains. These identities assume consistent discrete operators and boundary treatment and an exact linear solve; linear-solver residuals introduce corresponding errors. Ordinary cell-centered velocity and pressure corrections also follow the projection [1].

4 How this changes the amount of VOID

4.1 Position in the time-step sequence

The mainline cycle advances material transport before updating flow. With n denoting the last accepted state, the relevant sequence is [3, 4]

1. Advect materials using the previously accepted face velocities. Apply geometric-tracker boundedness and normalization corrections.
2. If thermal transport is active, update temperature and phase change; refresh the phase fractions supplied to flow.
3. Update flow properties and cell classifications. Compare the current VOID fraction to the stored accepted fraction to form $\delta\alpha_c$ in equation (3).
4. Advance momentum and project the face velocities with the extra source in equation (5).
5. Accept the flow/property state. The resulting face velocities drive the *next* material-transport update.

The response is therefore delayed through the split sequence. The projection does not revisit the material update that supplied its current VOID fractions. Initialization sets the stored fractions to the initial fractions and explicitly sets `void_delta_cc` to zero, so the collapse source is initially zero [2].

4.2 Preferential removal of VOID by the geometric tracker

For outward signed material flux volumes $\Phi_{m,cf}$, the tracker first accumulates fractions according to

$$\tilde{f}_{m,c} = f_{m,c}^{\text{start}} - \frac{1}{V_c} \sum_f \Phi_{m,cf}. \quad (11)$$

It then applies `enforce_bounded_vof` after each geometric subcycle. Following individual fraction cutoff/clipping operations, let

$$E_c = \sum_m \tilde{f}_{m,c} - 1, \quad a_c = \sum_{\text{VOID}} \tilde{f}_{m,c}. \quad (12)$$

If $E_c > 0$ and $a_c \geq E_c$, it removes the excess solely from VOID:

$$f_{\text{VOID},c}^{\text{corrected}} = \tilde{f}_{\text{VOID},c} \left(1 - \frac{E_c}{a_c} \right). \quad (13)$$

With the single VOID slot this is simply $a_c \mapsto a_c - E_c$. In this branch it leaves real-fluid fractions and their accumulated flux volumes unchanged. If VOID is insufficient, it removes all available VOID and uses `adjust_flux_all` for the remaining excess. Underfill also reaches the general adjustment path [5].

Equation (10) promotes overfilling, and equation (13) provides the associated VOID loss. For illustration, if material fluxes sum to the geometric swept volumes, an initially normalized cell transported for a subsequent interval Δt_{next} with these fixed face velocities has raw overfill

$$E_c = -\frac{\Delta t_{\text{next}}}{V_c} Q_c^{\text{new}} = q_c \frac{\Delta t_{\text{next}}}{\Delta t}. \quad (14)$$

This is q_c for equal step sizes. Geometry, flux corrections, material cutoffs, solid occupancy, and exhaustion of VOID can complicate the actual update. Equation (14) is an interpretation of the coupling, not a separate formula executed by the tracker.

The preferential VOID correction is present in geometric volume tracking regardless of the projection’s `void_collapse` flag. Wisp redistribution is another, separately controlled operation. The simple donor-cell tracker does not implement this geometric tracker’s boundedness and preferential-VOID-removal routine; it should not be assumed to provide the same collapse behavior merely because the projection flag is enabled [5, 6].

5 Consequences of the relaxation rule

5.1 Both signs drive contraction; an unchanged fraction does not

For $\alpha_c = 0.02$, $\lambda = 0.1$, and $\Delta t = 0.01$ seconds, the cap is $\lambda\alpha_c = 0.002$:

$\delta\alpha_c$	q_c	$Q_c^{\text{new}}/V_c \text{ (s}^{-1}\text{)}$
0	0	0
+0.001	0.001	−0.1
−0.001	0.001	−0.1
+0.01 or −0.01	0.002	−0.2

These are arithmetic examples, not run results. A stationary mixed pocket with unchanged fraction has no collapse source. Either an increase or a decrease in its fraction can activate the response. Even translation of a fragment through cells can produce the Eulerian fraction change used by the model. Pressure and inertia affect the preceding flow evolution, but the source contains no explicit pressure or inertial-force threshold.

5.2 The relaxation parameter is a cap, not a decay prescription

The model does not directly impose $\alpha^{n+1} = (1 - \lambda)\alpha^n$. The manual’s description of roughly $1/\lambda$ steps to collapse should be read as qualitative guidance. Only under additional assumptions—persistent saturation of the cap, equal step sizes, adequate VOID, and no other transport effects—would repeated removal resemble geometric decay. At $\lambda = 0.1$, that idealized law would leave $0.9^{10} \simeq 0.35$ of the original fraction after ten updates. It would not remove all VOID in ten steps. At $\lambda = 1$, complete removal is permitted by the cap but is not automatically requested, because $|\delta\alpha_c|$ remains a second limit.

The control is dimensionless and tied to time steps, rather than a fixed physical relaxation time. At fixed fractions, the added divergence scales as $1/\Delta t$. Because the resulting velocity is used in the next transport update, changing step sizes also changes the effective transported volume as in equation (14). The relaxation cap is thus not an unconditional bound on the amount removed in that next update.

6 Input controls, conservation, and limitations

The model is selected in the mainline FLOW namelist:

```
&FLOW
  void_collapse = T
```

```
void_collapse_relaxation = 0.1  
/
```

The default is disabled. The relaxation factor defaults to 0.1, and the namelist reader validates $0 \leq \lambda \leq 1$. The bridging parameter list uses `options/void-collapse` and `options/void-collapse-relaxation`. The projection applies the default relaxation value when it reads that list. Setting $\lambda = 0$ gives a zero additional source [7, 1, 8].

Conservation. VOID volume is deliberately not conserved by its normalization correction. Removing VOID alone from an overfilled cell need not change real-fluid mass. However, geometric tracking also uses cutoffs and cell-local flux adjustments. The source explicitly notes that those adjustments can make material flux volumes inconsistent across a shared face. Hence exact global real-fluid conservation is not guaranteed by the complete algorithm. The manual reports increased mass loss with larger relaxation factors (of order 0.5% in its experience) and improvement with more subcycling; those statements are empirical guidance, not error bounds or measurements made for this note [5, 8].

Global compatibility. The collapse source is local and has no global compensation mechanism. The discrete divergence theorem requires the sum of integrated divergences in a connected active region to equal its net boundary flux. Consequently, an entirely closed region with impermeable boundaries cannot satisfy a strictly negative total prescribed divergence. A free surface or open boundary can accommodate compensating motion. With only Neumann pressure conditions, pinning a pressure equation does not cure the compatibility condition of the original divergence equations; it permits a compensating defect in the modified equation. This is a mathematical limitation of applying a net source in that setting, not a failure reproduced in a run here.

Geometry and resolution. Eligibility depends on face adjacency and the existence of pure void cells. Refinement or mesh perturbation can therefore alter eligibility, even for similar physical interfaces. The model does not measure cavity size, reconstruct a closed void surface, or determine connectivity across the domain. Its local criterion should not be equated with identifying every physically trapped bubble.

Current test evidence. The repository’s `test/void` example constructs a restart with an injected VOID fraction and enables collapse. Its Python check bounds the maximum absolute change of the VOID field from the initial output by 10^{-4} . Although its comment speaks of preventing growth, the comparison bounds changes in either direction. It does not validate a collapse rate, the prescribed divergence, or liquid mass conservation [9]. Source inspection establishes the implemented rule; it does not establish accuracy or robustness for every geometry.

7 Source guide and document reproduction

The links below refer to the immutable mainline revision stated on the first page. Within a clone, for example, the core routine is available with

```
git show 344a7c3a:src/truchas/flow/flow_projection_type.F90
```

To rebuild this document with pdfLaTeX, run twice:

```
pdflatex -interaction=nonstopmode -halt-on-error void-collapse-model.tex
```

References

- [1] `flow_projection_type.F90`. `init`: defaults; `setup_solver`: collapse source, pressure matrix, and pinning; `velocity_fc_correct`: face-velocity correction.
- [2] `flow_props_type.F90`. `set_initial_state`, `update_cc`, `update_fc`, and `accept`: fraction history, cell/face classification, and state storage.
- [3] `drivers.F90`, main cycle; `vtrack_driver.F90`, `vtrack_update`; `flow_driver.F90`, `flow_step` and `flow_accept`.
- [4] `flow_type.F90`. `step`: property update, prediction, projection; `accept`: velocity history.
- [5] `geometric_volume_tracker_type.F90`. `flux_volumes`, `accumulate_volume`, `enforce_bounded_vof`, `adjust_flux_matl`, and `adjust_flux_all`.
- [6] `simple_volume_tracker_type.F90`. `flux_volumes`: donor-cell update.
- [7] `flow_namelist.F90`. Namelist defaults, relaxation bounds, and parameter-list translation.
- [8] Truchas reference manual, `FLOW` namelist. Entries `void_collapse` and `void_collapse_relaxation`.
- [9] `test/void/README.md`, `add-solid-void.py`, `test.inp`, and `test.py`. Restart preparation, input configuration, and existing test criterion.